

K&T CODE

Cotización Actualizada - Desarrollo KAILAB v2

Fecha: 21 de agosto de 2026

Cliente: KAILAB

Desarrollador: Keyner Steban Trillos Useche

Proyecto: KAILAB v2 - Plataforma Web E-commerce

Dominio: kailab.com.co

1. Contexto de la propuesta

A partir de la cotización presentada el 18 de agosto de 2026 y de las posteriores definiciones funcionales y técnicas, se establece el alcance actualizado para el desarrollo de **KAILAB v2**.

La propuesta inicial contemplaba:

- Sistema web base: **\$1.600.000 COP**
- Módulo bilingüe Español / Inglés: **\$200.000 COP**
- Total inicial: **\$1.800.000 COP**

Posteriormente el proyecto fue ampliado para incorporar:

- Comercio electrónico.
- Sistema de usuarios.
- Gestión de clientes.
- Pedidos.
- Trazabilidad de pagos.
- Dashboard administrativo personalizado.
- Reportes comerciales.
- Pagos mediante Wompi.
- Pagos con criptomonedas mediante NOWPayments.
- Microsoft Clarity.
- Sistema bilingüe.
- Ambiente privado de pruebas.
- Entrega del código.
- Propiedad tecnológica para KAILAB.

Esta ampliación llevó el proyecto a un valor de:

\$3.000.000 COP

Adicionalmente, se acuerda fortalecer la arquitectura tecnológica mediante una separación especializada de responsabilidades entre **Sanity** y **Supabase/PostgreSQL**,

manteniendo ambos sistemas completamente integrados dentro del funcionamiento de KAILAB v2.

Valor adicional de arquitectura

\$400.000 COP

Inversión total actualizada

\$3.400.000 COP

Tres millones cuatrocientos mil pesos colombianos.

2. Objetivo del proyecto

Desarrollar la base tecnológica de **KAILAB v2** como una plataforma web moderna, escalable, bilingüe y preparada para comercio electrónico.

El sistema permitirá manejar:

- Productos.
- Presentaciones.
- Lotes.
- Certificados de análisis.
- Usuarios.
- Clientes.
- Carrito.
- Checkout como invitado.
- Pedidos.
- Ítems de pedido.
- Intentos de pago.
- Wompi.
- NOWPayments.
- Estados de pago.
- Área privada del cliente.
- Dashboard administrativo.
- Reportes.
- Analítica de comportamiento.

La solución será desarrollada como una **arquitectura integrada**, donde Sanity y Supabase/PostgreSQL trabajarán conjuntamente dentro de una sola aplicación.

La primera entrega se realizará en un ambiente privado de pruebas —**staging**—.

El código será desarrollado como una base reutilizable para las siguientes fases de KAILAB v2 y no como un prototipo temporal.

3. Arquitectura tecnológica

La solución utilizará principalmente:

Next.js

Framework principal para:

- Sitio web.
- Interfaces.
- Carrito.
- Checkout.
- Sistema de usuarios.
- Área privada.
- Dashboard.
- Lógica de negocio.
- Comunicación con Sanity.
- Comunicación con Supabase/PostgreSQL.
- Integración entre ambas arquitecturas.
- Validaciones del lado del servidor.
- Integraciones de pago.
- APIs y procesos internos necesarios.

Vercel

Infraestructura destinada a:

- Despliegue.
- Ambiente de staging.
- Aplicación web.
- Procesos del lado del servidor.
- Posterior ambiente de producción.

Sanity

Sanity se mantendrá como plataforma principal para la administración de contenido y catálogo.

Será utilizado principalmente para:

- Productos.
- Presentaciones.

- Imágenes.
- Precios de catálogo.
- SKUs.
- Descripciones.
- Información técnica.
- Lotes.
- Certificados de análisis – COAs.
- Traducciones.
- Contenido bilingüe.
- Contenido comercial.
- Contenido editorial.
- Metadatos.

Supabase / PostgreSQL

Supabase/PostgreSQL será utilizado como plataforma para la información de naturaleza transaccional.

Permitirá almacenar y relacionar:

- Clientes.
- Perfiles relacionados con usuarios.
- Pedidos.
- Ítems de pedidos.
- Cantidades.
- Precios históricos de compra.
- Totales.
- Intentos de pago.
- Métodos de pago.
- Estados de pago.
- Estados operativos.
- Referencias de Wompi.
- Referencias de NOWPayments.
- Eventos de proveedores de pago.
- Datos necesarios para trazabilidad.
- Información transaccional utilizada en reportes.

4. Arquitectura unificada Sanity + Supabase/PostgreSQL

Sanity y Supabase/PostgreSQL no funcionarán como dos proyectos independientes o aislados.

Ambas tecnologías deberán quedar conectadas mediante la aplicación desarrollada en Next.js, formando una arquitectura única para KAILAB v2.

La separación responde a la naturaleza de la información:

Sanity

Administrará principalmente:

Contenido y catálogo

- Productos.
- Presentaciones.
- Imágenes.
- Información técnica.
- Lotes.
- COAs.
- Traducciones.
- Contenido comercial.
- Contenido editorial.

Supabase / PostgreSQL

Administrará principalmente:

Información transaccional

- Clientes.
- Pedidos.
- Ítems.
- Intentos de pago.
- Estados.
- Métodos de pago.
- Referencias.
- Eventos.
- Historial transaccional.

Next.js

Actuará como la capa encargada de **unir ambas arquitecturas**.

La aplicación deberá consultar, relacionar, validar y procesar información proveniente de Sanity y PostgreSQL según corresponda.

Esta integración será necesaria para garantizar el correcto funcionamiento en línea de:

- Catálogo.
- Carrito.
- Checkout.
- Pedidos.
- Pagos.
- Usuarios.
- Área privada.

- Dashboard.
- Reportes.

Para el usuario final y para el administrador, KAILAB deberá funcionar como **una sola plataforma**, independientemente de que internamente utilice dos sistemas especializados de almacenamiento.

5. Relación entre catálogo y transacciones

Los productos continuarán siendo administrados en Sanity.

Al momento de generar una compra, el sistema podrá utilizar información proveniente de Sanity y posteriormente registrar en PostgreSQL la información transaccional correspondiente.

Cada ítem de pedido podrá conservar:

- Identificador del producto en Sanity.
- Identificador de presentación.
- SKU.
- Nombre del producto.
- Presentación.
- Cantidad.
- Precio registrado al momento de la compra.
- Subtotal.

Esto permitirá que las modificaciones futuras realizadas al catálogo no alteren el histórico de pedidos previamente realizados.

6. Producto y presentaciones

La primera etapa incorporará un producto real de KAILAB con dos presentaciones comerciales.

Por ejemplo:

- 5 mg.
- 10 mg.

Cada presentación podrá manejar:

- Nombre.
- Cantidad.

- Precio.
- SKU.
- Estado.
- Información relacionada.

La estructura quedará preparada para incorporar posteriormente el catálogo completo.

7. Tarjeta y página de producto

Se desarrollarán los componentes necesarios para presentar y comercializar el producto.

Tarjeta de producto

Componente reutilizable para:

- Home.
- Catálogo.
- Categorías.
- Resultados de búsqueda.
- Productos relacionados.

Página individual de producto

Incluirá:

- Nombre.
- Imágenes.
- Descripción.
- Selector de presentación.
- Precio.
- SKU.
- Información comercial.
- Información relacionada con calidad y lotes.
- Acción para agregar al carrito.

El contenido será administrable desde Sanity.

8. Lotes y certificados de análisis - COA

La primera etapa incluirá:

- Un lote real.
- Código o número de lote.

- Producto relacionado.
- Información básica del lote.
- Certificado de análisis.
- Documento COA.
- Visualización del certificado.

La arquitectura quedará preparada para incorporar posteriormente nuevos lotes y certificados.

9. Carrito de compras

Se implementará un carrito funcional que permita:

- Agregar productos.
- Seleccionar presentación.
- Definir cantidad.
- Modificar cantidad.
- Eliminar productos.
- Consultar precio.
- Consultar subtotal.
- Consultar total.
- Conservar temporalmente el carrito durante la navegación.

Antes de generar un pedido se realizarán validaciones desde servidor.

La aplicación deberá consultar la información necesaria desde Sanity y relacionarla correctamente con la operación que posteriormente será registrada en PostgreSQL.

10. Checkout como invitado

La plataforma permitirá finalizar una compra sin obligar al comprador a crear previamente una cuenta.

El flujo contemplará:

1. Revisión del carrito.
2. Registro de información del comprador.
3. Validación del carrito y productos.
4. Consulta de información correspondiente desde Sanity.
5. Validación desde servidor.
6. Creación o asociación del cliente.
7. Creación del pedido en PostgreSQL.
8. Registro de ítems.

9. Selección del método de pago.
10. Registro del intento de pago.
11. Inicio de la transacción.
12. Recepción o consulta del resultado.
13. Actualización del estado del pedido.

Los pedidos e información transaccional quedarán registrados en PostgreSQL/Supabase.

11. Integración con Wompi

Se implementará Wompi inicialmente en ambiente Sandbox.

Incluye:

- Generación de referencias.
- Inicio del proceso de pago.
- Asociación entre pedido y transacción.
- Registro de intentos.
- Recepción de eventos.
- Actualización de estados.
- Pagos aprobados.
- Pagos rechazados.
- Pagos pendientes.
- Trazabilidad.

La información transaccional correspondiente será registrada en PostgreSQL/Supabase.

12. Integración con NOWPayments

Se integrará NOWPayments como alternativa de pago mediante criptomonedas.

Contemplará:

- Integración técnica.
- Opción dentro del checkout.
- Creación de solicitudes de pago.
- Asociación entre pedido y pago.
- Registro del proveedor.
- Registro de identificadores.
- Registro de intentos.
- Consulta o recepción de estados.
- Actualización del pedido.
- Estados exitosos.

- Estados pendientes.
- Estados fallidos.
- Información para trazabilidad.

La información transaccional correspondiente será almacenada en PostgreSQL.

13. Prevención de duplicados e idempotencia

La lógica de pagos será desarrollada buscando evitar:

- Pedidos duplicados.
- Pagos duplicados.
- Intentos duplicados.
- Eventos procesados varias veces.
- Actualizaciones incorrectas.
- Procesamiento múltiple de una misma operación.

La arquitectura PostgreSQL permitirá utilizar:

- Identificadores únicos.
- Restricciones.
- Relaciones.
- Referencias.
- Registro de eventos procesados.
- Validaciones desde servidor.

Esto aplicará a:

- Wompi.
 - NOWPayments.
-

14. Gestión de pedidos

PostgreSQL/Supabase conservará principalmente el historial transaccional.

Cada pedido podrá almacenar:

- Número de pedido.
- Cliente.
- Fecha.
- Subtotal.
- Total.
- Método de pago.

- Estado del pago.
- Estado operativo.
- Referencia de transacción.

Cada pedido podrá contener uno o varios ítems con:

- Producto.
 - Referencia hacia Sanity.
 - Presentación.
 - SKU.
 - Cantidad.
 - Precio histórico.
 - Subtotal.
-

15. Intentos de pago

Los intentos de pago podrán registrarse independientemente.

Ejemplo:

Pedido #1005

- Intento 1 – Wompi: rechazado.
- Intento 2 – Wompi: aprobado.

Otro ejemplo:

Pedido #1006

- NOWPayments.
- Estado inicial: pendiente.
- Estado posterior: confirmado.

Esto permitirá conservar trazabilidad del proceso comercial.

16. Sistema de usuarios

Se implementará:

- Registro.
- Inicio de sesión.
- Cierre de sesión.
- Manejo seguro de sesiones.

- Recuperación de acceso.
- Asociación entre usuario y cliente.
- Asociación entre cliente y pedidos.
- Área privada.

Los datos transaccionales relacionados con usuarios y clientes estarán integrados dentro de la arquitectura Supabase/PostgreSQL.

El checkout como invitado continuará disponible.

17. Área privada del cliente

Los clientes registrados podrán acceder a:

Mis pedidos

- Número de pedido.
- Fecha.
- Productos.
- Presentaciones.
- Cantidades.
- Total.
- Método de pago.
- Estado del pedido.
- Estado del pago.

Detalle de pedido

Permitirá consultar individualmente la información correspondiente.

18. Dashboard administrativo unificado

Se desarrollará un dashboard administrativo personalizado para KAILAB.

Será responsive y funcionará desde:

- Computador.
- Tablet.
- Celular.

El dashboard será **una sola interfaz administrativa**, aunque internamente utilice información proveniente de dos arquitecturas.

Podrá consumir:

Desde Sanity

- Productos.
- Presentaciones.
- Contenido.
- Lotes.
- COAs.

Desde PostgreSQL/Supabase

- Clientes.
- Pedidos.
- Pagos.
- Intentos.
- Estados.
- Datos comerciales.

La integración entre ambas fuentes deberá realizarse de forma transparente para el administrador.

19. Dashboard - Vista general

Podrá mostrar:

- Total de pedidos.
 - Pedidos recientes.
 - Pedidos por estado.
 - Ventas registradas.
 - Pagos aprobados.
 - Pagos pendientes.
 - Pagos rechazados.
 - Número de clientes.
 - Actividad comercial.
 - Distribución de pagos según método.
-

20. Dashboard - Pedidos

Permitirá consultar:

- Número.
- Cliente.

- Fecha.
- Valor.
- Método de pago.
- Estado.
- Estado del pago.
- Productos.
- Presentaciones.
- Cantidades.
- Referencias.
- Intentos de pago.

Estados básicos:

- Nuevo.
- En proceso.
- Completado.
- Cancelado.

21. Dashboard - Clientes

Podrá mostrar:

- Nombre.
- Correo.
- Fecha de registro.
- Cantidad de pedidos.
- Historial básico de compras.
- Información comercial relacionada.

22. Reportes administrativos

La primera etapa permitirá consultar información relacionada con:

- Pedidos.
- Clientes.
- Ventas.
- Estados.
- Pagos.
- Métodos de pago.

La arquitectura quedará preparada para futuras funciones como:

- Rangos avanzados de fechas.

- Productos más vendidos.
 - Ticket promedio.
 - Clientes recurrentes.
 - Comparativos.
 - Exportaciones.
 - Inventario.
 - Reportes financieros avanzados.
-

23. Microsoft Clarity

Se integrará Microsoft Clarity para analizar:

- Grabaciones de sesiones.
- Mapas de calor.
- Clics.
- Desplazamiento.
- Interacción.
- Puntos de frustración.
- Página de producto.
- Comportamiento previo al checkout.
- Posibles puntos de abandono.

La cuenta correspondiente quedará bajo control de KAILAB.

24. Sistema bilingüe Español / Inglés

La propuesta incluye:

- Español.
- Inglés.
- Selector de idioma.
- Navegación por idioma.
- Estructura de rutas.
- Campos bilingües en Sanity.
- Producto bilingüe.
- Página de producto.
- Carrito.
- Checkout.
- Login.
- Área privada.
- Interfaz general.
- Metadatos correspondientes.

Traducciones

K&T CODE realizará e incorporará las traducciones al inglés correspondientes al contenido de esta etapa.

25. Ambiente privado de pruebas - Staging

Se configurará staging para:

- Revisar interfaces.
 - Crear usuarios.
 - Realizar pedidos.
 - Probar Wompi.
 - Probar NOWPayments.
 - Consultar clientes.
 - Consultar pedidos.
 - Revisar dashboard.
 - Validar contenido bilingüe.
 - Realizar pruebas móviles.
 - Validar la conexión Sanity/Supabase.
 - Validar procesos de lectura y escritura.
 - Corregir defectos antes de producción.
-

26. Integración para correcto funcionamiento en línea

Dentro del valor contratado se incluye el trabajo necesario para que **Sanity y Supabase/PostgreSQL funcionen conjuntamente dentro de la plataforma en línea.**

Esto comprende:

- Conexión entre Next.js y Sanity.
- Conexión entre Next.js y Supabase/PostgreSQL.
- Relación entre productos y pedidos.
- Relación entre presentaciones e ítems de pedido.
- Validación de datos desde servidor.
- Flujo de información catálogo → carrito → checkout → pedido.
- Flujo pedido → intento de pago → proveedor de pago.
- Recepción de estado → PostgreSQL → dashboard.
- Consulta del historial desde el área privada.
- Consulta de operaciones desde el dashboard.
- Manejo adecuado de identificadores entre ambos sistemas.

La separación tecnológica no deberá afectar negativamente la experiencia del cliente ni del administrador.

El resultado deberá funcionar como **un único sistema conectado**.

27. Código y arquitectura reutilizables

Serán reutilizables:

- Componentes.
 - Modelos de Sanity.
 - Modelo PostgreSQL.
 - Integraciones.
 - Productos.
 - Presentaciones.
 - Lotes.
 - COAs.
 - Usuarios.
 - Clientes.
 - Carrito.
 - Checkout.
 - Pedidos.
 - Ítems.
 - Intentos de pago.
 - Eventos.
 - Wompi.
 - NOWPayments.
 - Dashboard.
 - Bilingüe.
 - Analítica.
 - Capa de integración Sanity/Supabase.
-

28. Valor adicional por arquitectura integrada

La implementación de la separación y posterior unificación funcional de las dos capas representa un valor adicional de:

\$400.000 COP

Este valor contempla:

- Configuración de Supabase/PostgreSQL.
- Diseño del modelo transaccional.

- Clientes.
- Pedidos.
- Ítems.
- Intentos de pago.
- Estados.
- Eventos.
- Relaciones.
- Restricciones.
- Integración desde Next.js.
- Comunicación con Sanity.
- Adaptación del checkout.
- Adaptación del sistema de pedidos.
- Adaptación de Wompi.
- Adaptación de NOWPayments.
- Adaptación del área privada.
- Adaptación del dashboard.
- Pruebas de integración.
- Validación del flujo completo en línea.
- Documentación básica de arquitectura.

Este valor no corresponde únicamente a crear una base de datos adicional.

Corresponde a desarrollar la **integración funcional entre Sanity, Supabase/PostgreSQL y Next.js**, de manera que las diferentes capas trabajen como una única plataforma.

29. Propiedad y control de KAILAB

Las cuentas principales permanecerán bajo control de KAILAB.

Incluye:

- Repositorio.
- Vercel.
- Sanity.
- Supabase.
- PostgreSQL.
- Wompi.
- NOWPayments.
- Microsoft Clarity.
- Dominio.
- Ambientes.

Una vez completado el pago total, el código desarrollado específicamente para KAILAB será propiedad de KAILAB.

Se entregarán:

- Código fuente.
- Repositorio.
- Esquemas de Sanity.
- Modelo de PostgreSQL.
- Configuraciones correspondientes.
- Integraciones.
- Accesos acordados.
- Resumen técnico.

30. Documentación

Se entregará un resumen técnico que incluya:

- Arquitectura general.
- Next.js.
- Sanity.
- Supabase.
- PostgreSQL.
- Integración entre Sanity y PostgreSQL.
- Estructura de datos.
- Relación catálogo/transacciones.
- Vercel.
- Wompi.
- NOWPayments.
- Microsoft Clarity.
- Ambientes.
- Configuraciones.
- Cuentas utilizadas.
- Costos externos.
- Consideraciones para siguientes etapas.

31. Inversión total

Desarrollo funcional y tecnológico

\$3.000.000 COP

Arquitectura integrada Sanity + Supabase/PostgreSQL

\$400.000 COP

INVERSIÓN TOTAL

\$3.400.000 COP

Tres millones cuatrocientos mil pesos colombianos.

Este valor incluye el desarrollo, integración y pruebas correspondientes al alcance descrito en esta propuesta.

32. Justificación del incremento

La propuesta inicial correspondía a:

\$1.800.000 COP

La ampliación de comercio electrónico, usuarios, pagos, administración y demás funcionalidades llevó el proyecto a:

\$3.000.000 COP

La incorporación de la arquitectura integrada:

Sanity + Supabase/PostgreSQL

representa:

\$400.000 COP adicionales.

Total final

\$3.400.000 COP

El incremento adicional contempla principalmente:

- Nueva capa PostgreSQL.
 - Supabase.
 - Modelado transaccional.
 - Relaciones.
 - Integridad de datos.
 - Integración con Sanity.
 - Integración desde Next.js.
 - Adaptación de pedidos.
 - Adaptación de pagos.
 - Adaptación del dashboard.
 - Adaptación del área privada.
 - Pruebas integrales.
 - Documentación.
-

33. Forma de pago

50% al iniciar

\$1.700.000 COP

50% contra entrega y aceptación en staging

\$1.700.000 COP

El segundo pago se realizará una vez completado y validado el alcance contratado y corregidos los defectos críticos identificados durante la revisión.

34. Datos para el pago

Nequi: 3133087069

Bancolombia - Cuenta de Ahorros:
91290318578

35. Plazo del proyecto

4 a 6 semanas

El tiempo estimado para el desarrollo completo de esta etapa será de **cuatro (4) a seis (6) semanas**.

Este plazo contempla:

- Arquitectura.
- Configuración de Sanity.
- Configuración de Supabase/PostgreSQL.
- Integración entre ambas tecnologías.
- Producto.
- Comercio electrónico.
- Sistema de usuarios.
- Área privada.
- Pedidos.
- Pagos.
- Dashboard.
- Bilingüe.
- Analítica.
- Staging.
- Pruebas.
- Corrección de defectos correspondientes al alcance.

- Documentación.
- Entrega técnica.

El plazo comenzará a contabilizarse desde:

- Aceptación de la propuesta.
- Recepción del anticipo.
- Entrega de accesos.
- Entrega de información inicial necesaria.

Fecha posible de inicio

A partir del **24 de agosto de 2026**.

Los períodos de espera causados por falta de información, accesos, credenciales o aprobaciones de KAILAB no serán contabilizados dentro del plazo de 4 a 6 semanas.

36. Hitos

Hito 1 - Arquitectura

- Repositorio.
- Vercel.
- Staging.
- Sanity.
- Supabase/PostgreSQL.
- Modelos.
- Relaciones.
- Sistema bilingüe.
- Integración inicial entre arquitecturas.

Hito 2 - Producto y calidad

- Producto.
- Presentaciones.
- Tarjeta.
- Página de producto.
- Lote.
- COA.
- Traducciones.

Hito 3 - Comercio electrónico

- Carrito.
- Checkout.
- Clientes.

- Pedidos.
- Ítems.
- Intentos.
- Wompi.
- NOWPayments.
- Eventos.
- Idempotencia.

Hito 4 - Usuarios y administración

- Registro.
- Login.
- Recuperación.
- Área privada.
- Historial.
- Dashboard.
- Clientes.
- Pedidos.
- Reportes.
- Integración Sanity/Supabase.

Hito 5 - Analítica y entrega

- Microsoft Clarity.
- Pruebas integrales.
- Pruebas de comunicación entre arquitecturas.
- Pruebas de pagos.
- Corrección de defectos.
- Documentación.
- Entrega de código.
- Entrega de accesos.
- Aceptación en staging.

37. Criterios de aceptación

La etapa se considerará correctamente entregada cuando:

Infraestructura

- Staging esté operativo.
- Sanity esté configurado.
- Supabase/PostgreSQL esté configurado.
- Next.js pueda comunicarse correctamente con ambos sistemas.

Integración

- Sanity y Supabase/PostgreSQL estén correctamente conectados con la aplicación.
- Los productos de Sanity puedan utilizarse dentro del flujo de compra.
- Los pedidos generados desde productos administrados en Sanity queden registrados correctamente en PostgreSQL.
- Los ítems mantengan referencias y datos históricos necesarios.
- El dashboard pueda consultar la información correspondiente desde ambas fuentes.
- El área privada pueda consultar pedidos transaccionales almacenados en PostgreSQL.
- El sistema funcione como una única plataforma desde la perspectiva del usuario.

Comercio electrónico

- Carrito funcione.
- Checkout como invitado funcione.
- Pedido quede almacenado.
- Ítems queden registrados.
- Precios sean validados desde servidor.

Pagos

- Wompi Sandbox funcione.
- NOWPayments esté integrado.
- Intentos queden registrados.
- Métodos de pago puedan identificarse.
- Estados puedan actualizarse.
- Eventos repetidos no produzcan procesamiento duplicado.

Usuarios

- Registro funcione.
- Login funcione.
- Recuperación de acceso esté implementada.
- Área privada esté operativa.
- Cliente pueda consultar sus pedidos.

Administración

- Dashboard esté operativo.
- Sea usable desde computador, tablet y celular.
- Pedidos puedan consultarse.
- Clientes puedan consultarse.
- Pagos puedan identificarse.
- Reportes básicos funcionen.

Contenido

- Español e inglés estén habilitados.
- Producto esté disponible en ambos idiomas.
- Lote y COA estén relacionados.

Entrega

- Microsoft Clarity esté integrado.
- KAILAB tenga acceso al repositorio.
- KAILAB tenga control de Sanity.
- KAILAB tenga control de Supabase.
- Código fuente sea entregado.
- Esquemas de Sanity sean entregados.
- Modelo PostgreSQL sea entregado/documentado.
- Defectos críticos del alcance hayan sido corregidos.

38. Garantía técnica

Se incluyen **30 días calendario de garantía técnica** posteriores a la aceptación.

Incluye corrección de defectos reproducibles relacionados directamente con el alcance contratado.

No incluye:

- Nuevas funcionalidades.
- Nuevas páginas.
- Cambios de alcance.
- Nuevas integraciones.
- Rediseños.
- Migraciones masivas.
- Funciones adicionales.
- Modificaciones realizadas por terceros.
- Cambios de proveedores externos.

39. Costos recurrentes y servicios externos

Los **\$3.400.000 COP** corresponden al desarrollo e implementación.

Los costos futuros cobrados directamente por proveedores externos serán asumidos por KAILAB cuando corresponda.

Podrán existir costos asociados a:

- Vercel.
- Sanity.
- Supabase.
- Wompi.
- NOWPayments.
- Otros servicios utilizados posteriormente.

Las comisiones transaccionales no están incluidas dentro del valor del desarrollo.

40. Responsabilidades de KAILAB

KAILAB deberá proporcionar:

- Acceso al dominio.
- Logo.
- Manual de marca si existe.
- Fotografías.
- Información técnica y comercial.
- Presentaciones.
- Precios.
- SKUs.
- Información del lote.
- COA.
- Credenciales Sandbox de Wompi.
- Cuenta y credenciales de NOWPayments.
- Cuenta de Supabase o autorización para su creación/configuración.
- Contenido original.
- Correos propietarios de las cuentas.
- Retroalimentación.
- Aprobaciones.

Las traducciones al inglés correspondientes a esta etapa serán realizadas por K&T CODE.

41. Exclusiones

No se incluye:

- Catálogo completo.
 - Migración masiva.
 - Histórico completo de COAs.
 - Traducción masiva futura.
- 
- A decorative graphic in the bottom right corner consisting of a series of black rectangles of increasing size, creating a staircase effect.

- Facturación electrónica.
 - Integraciones logísticas.
 - CRM.
 - ERP.
 - Aplicación móvil nativa.
 - Sincronizaciones o integraciones adicionales no contempladas.
 - Reportes financieros avanzados.
 - Funcionalidades no expresamente indicadas.
-

42. Consideración final de arquitectura

KAILAB v2 utilizará una arquitectura especializada pero completamente integrada.

Sanity

Será responsable principalmente del **contenido y catálogo**.

Supabase/PostgreSQL

Será responsable principalmente de la **información transaccional**.

Next.js

Será responsable de **unificar ambas arquitecturas**, implementar la lógica de negocio y permitir que toda la plataforma funcione correctamente en línea.

Esto significa que Sanity y Supabase/PostgreSQL no serán considerados dos sistemas aislados.

Ambos formarán parte de una **única arquitectura tecnológica de KAILAB v2**.

La finalidad de esta separación e integración será obtener:

- Mejor organización de datos.
 - Mayor trazabilidad.
 - Separación entre contenido y transacciones.
 - Mejor capacidad de crecimiento.
 - Mayor facilidad para futuras integraciones.
 - Conservación del histórico comercial.
 - Administración centralizada.
 - Correcto funcionamiento de la plataforma en línea.
-

K&T CODE desarrollará esta etapa con el objetivo de entregar a KAILAB una base tecnológica propia, escalable, bilingüe, reutilizable e integrada.

La solución utilizará **Sanity y Supabase/PostgreSQL de forma conjunta**, manteniendo cada tecnología enfocada en el tipo de información para el cual resulta más adecuada, pero conectándolas mediante la aplicación para garantizar el funcionamiento integral de KAILAB v2.

Inversión total: \$3.400.000 COP

Plazo total del proyecto: 4 a 6 semanas

Cordialmente,

Keyner Steban Trillos Useche
K&T CODE

Correo: contactoktweb@gmail.com
Teléfono: +57 311 616 0057
Web: www.kytcode.lat